Lab 2

Terraform Deployment & Verification Report

1. Overview

This document describes the Terraform-based Lab 2 stack deployed on Oracle Cloud Infrastructure (OCI) and records the verification steps carried out after `terraform apply`, with supporting console and terminal evidence.

The stack provisions a segmented network with a public-facing load balancer and bastion host, and a private application tier reachable only through the bastion, with its data backed by OCI File Storage Service (FSS).

1.1 Architecture Diagram

The diagram below summarizes the network layout, traffic flows, security list rules, and route tables described in this document.

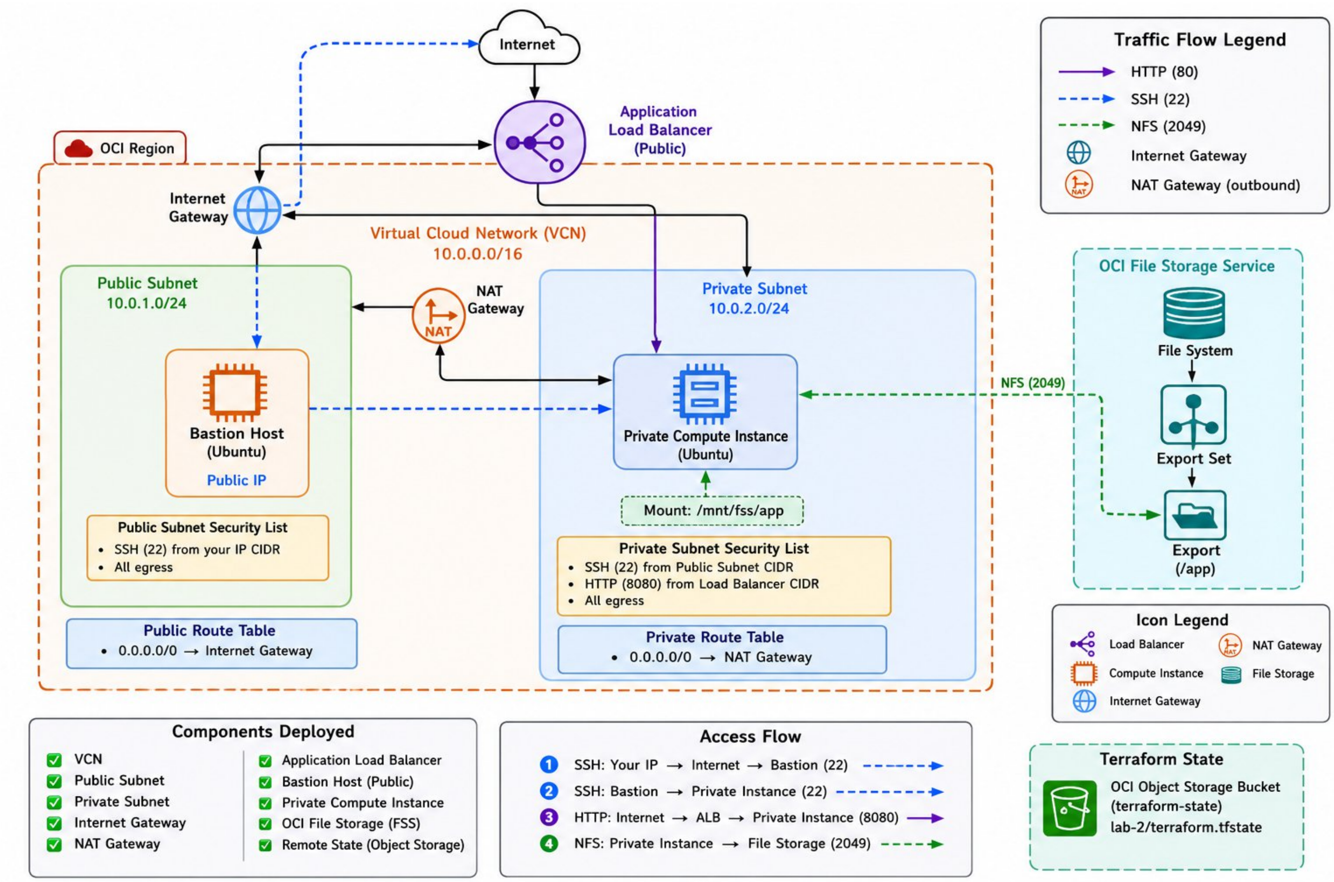

Figure 1 — Lab 2 architecture: public/private subnets, ALB, bastion, NAT/Internet gateways, FSS, and the Terraform state backend.

1.2 Resources Deployed

- 1 Virtual Cloud Network (VCN)
- 1 public subnet and 1 private subnet
- 1 Internet Gateway and 1 NAT Gateway
- 1 public Application Load Balancer (ALB)
- 1 bastion host in the public subnet
- 1 private compute instance in the private subnet
- 1 OCI File Storage Service (FSS) file system mounted on the private instance
- Remote Terraform state stored in an OCI Object Storage backend

1.2 Repository Layout

File / Path	Purpose
backend.tf	Declares the OCI Object Storage Terraform backend.
main.tf	Passes root variables into the lab2 module.
outputs.tf	Exposes the key values after apply (IPs, IDs).
provider.tf	Configures the OCI Terraform provider.
terraform.tfvars	Real environment values for the lab (compartment, region, keys, CIDRs).
variables.tf	Root input variable declarations.
modules/lab2/data.tf	Looks up availability domains.
modules/lab2/locals.tf	Reusable names and cloud-init/bootstrap values.
modules/lab2/vcn.tf	VCN, Internet/NAT gateways, route tables.
modules/lab2/subnet.tf	Security lists and subnets.
modules/lab2/storage.tf	File Storage Service resources.
modules/lab2/compute.tf	Private instance and bastion host.
modules/lab2/load_balancer.tf	Application Load Balancer and backend set.
modules/lab2/outputs.tf	Module-level outputs.
modules/lab2/variables.tf	Module input variable declarations.

2. Prerequisites

- Terraform installed locally
- OCI CLI installed and configured
- A valid OCI API key profile in ~/.oci/config
- A valid tenancy OCID and compartment OCID
- An SSH public key for instance access
- IAM permission to create networking, compute, load balancer, and file storage resources
- An OCI Object Storage bucket to hold the Terraform state file

Before running the lab, the following values must be reviewed in terraform.tfvars: compartment_id, tenancy_ocid, region, ssh_public_key, and bastion_ssh_source_cidr. Placeholder values must be replaced. The Ubuntu image is resolved automatically via the OCI image data source, so no image OCID needs to be supplied manually.

3. Deployment Procedure

Step 1 — Confirm the backend bucket exists

Before running Terraform, confirm the Object Storage namespace and bucket exist and that the OCI profile in use can read/write objects in that bucket. The state bucket used for this lab is shown below.

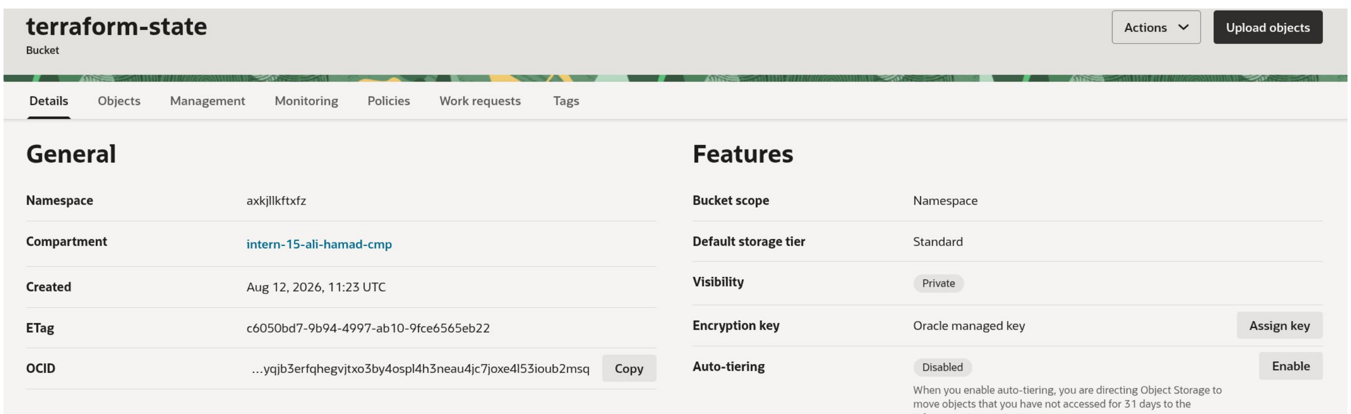

Figure 2 — Object Storage bucket "terraform-state" used for the OCI Terraform backend (namespace axkjllkftxfz, compartment intern-15-ali-hamad-cmp).

Step 2 — Format the configuration

```
terraform fmt -recursive
```

Step 3 — Initialize Terraform with the OCI backend

The backend block is declared in backend.tf, but the backend connection settings are supplied at init time:

```
terraform init \    -backend-config="namespace=YOUR_NAMESPACE" \    -backend-config="bucket=terraform-state" \    -backend-config="key=lab-2/terraform.tfstate" \    -backend-config="region=me-jeddah-1" \    -backend-config="auth=APIKey" \    -backend-config="config_file_profile=DEFAULT"
```

If Terraform prompts to migrate local state to remote state, confirm the migration.

Step 4 — Validate

```
terraform validate
```

Confirms the configuration is syntactically correct and that module references resolve properly.

Step 5 — Plan

```
terraform plan
```

Confirm the plan includes: the VCN and gateways, both subnets, security lists and route tables, the FSS mount target/export set/file system/export, the private instance, the bastion host, and the ALB with its backend set.

Step 6 — Apply

```
terraform apply
```

Type yes when prompted to confirm.

Step 7 — Review outputs

```
terraform output
```

Key outputs: vcn_id, private_instance_id, private_instance_ip, bastion_instance_id, bastion_public_ip, load_balancer_public_ips.

4. Deployed Infrastructure — Console Evidence

4.1 Virtual Cloud Network and Subnets

The VCN (main-vcn) contains one public and one private regional subnet, matching the design in vcn.tf and subnet.tf.

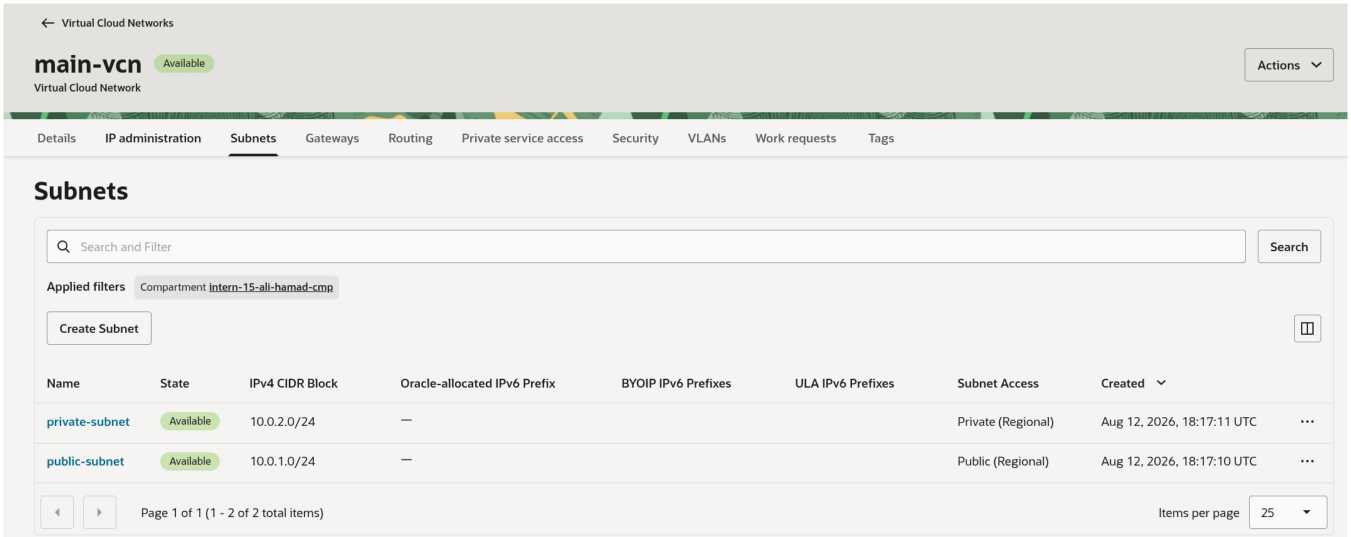

Figure 3 — main-vcn subnets: public-subnet (10.0.1.0/24, Public) and private-subnet (10.0.2.0/24, Private).

4.2 Compute Instances

Two instances were created: the public bastion host and the private application instance, both running Canonical Ubuntu 22.04 in availability domain AD-1, fault domain FD-1, region me-jeddah-1.

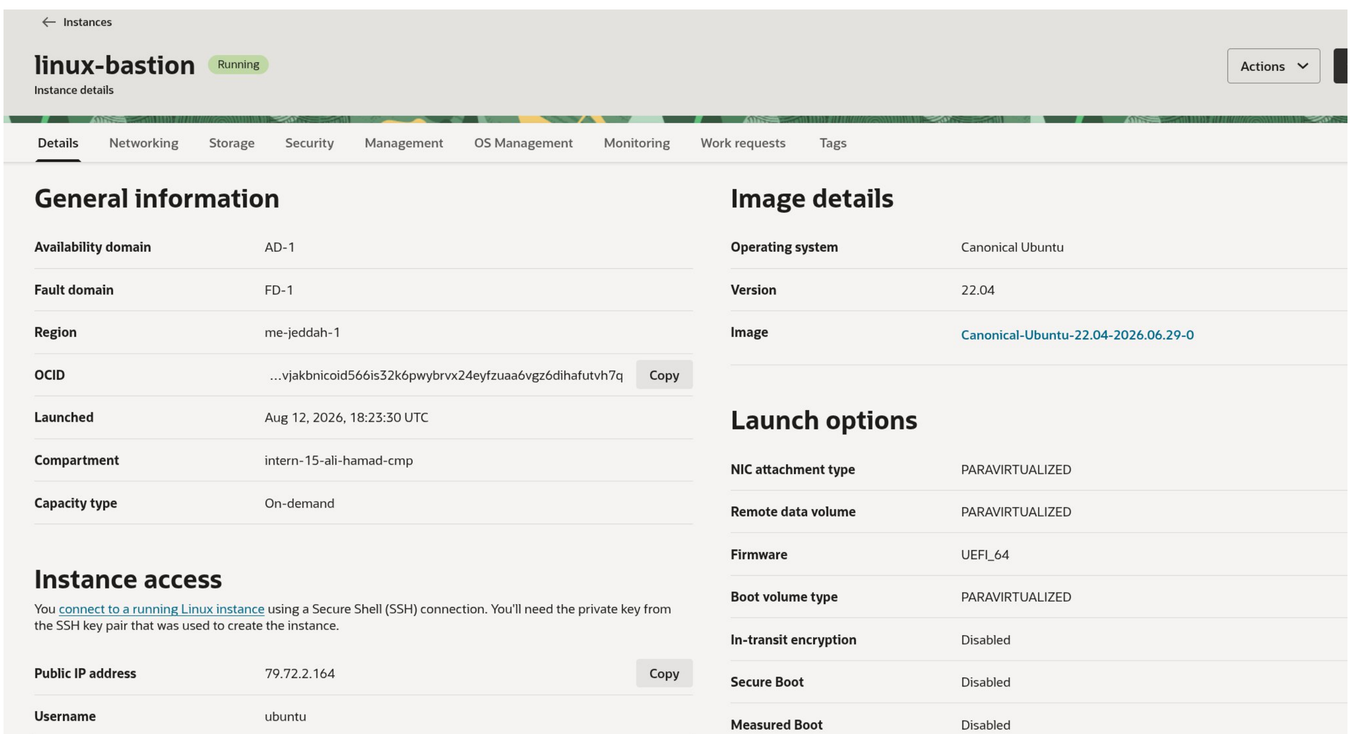

Figure 4 — linux-bastion instance details: state Running, public IP 79.72.2.164, username ubuntu.

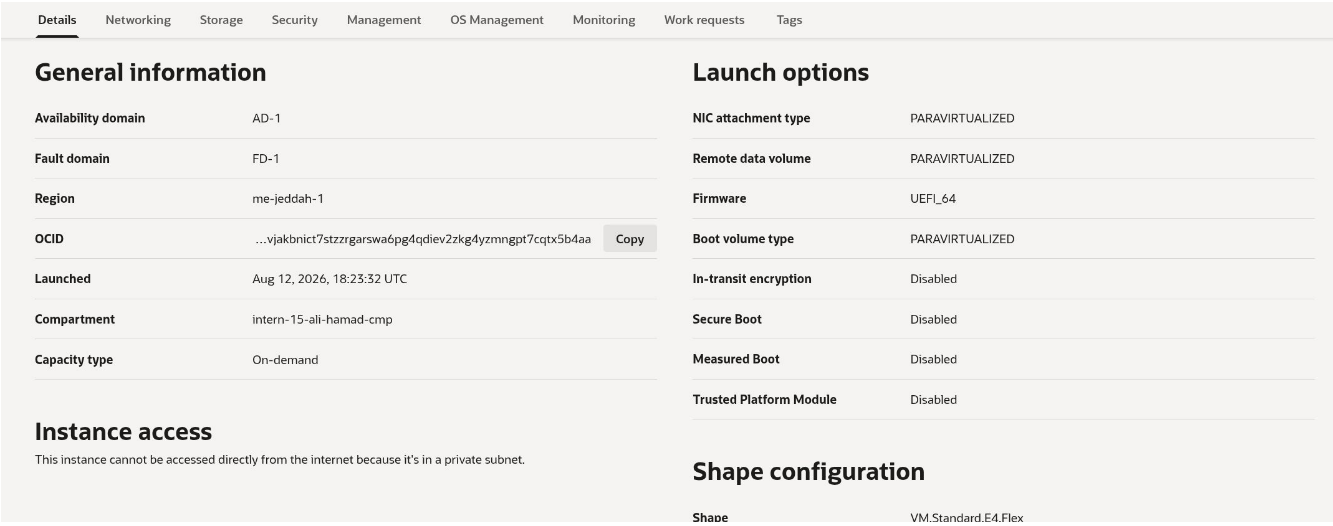

Figure 5 — Private instance details: VM.Standard.E4.Flex shape, no public IP ("cannot be accessed directly from the internet because it's in a private subnet").

4.3 Private Subnet Security List

Ingress rules on lab-private-security-list allow ICMP echo, application traffic (TCP/8080) and SSH (TCP/22) from the public subnet (10.0.1.0/24), and NFS (TCP/2049) from within the private subnet (10.0.2.0/24) itself for the FSS mount target.

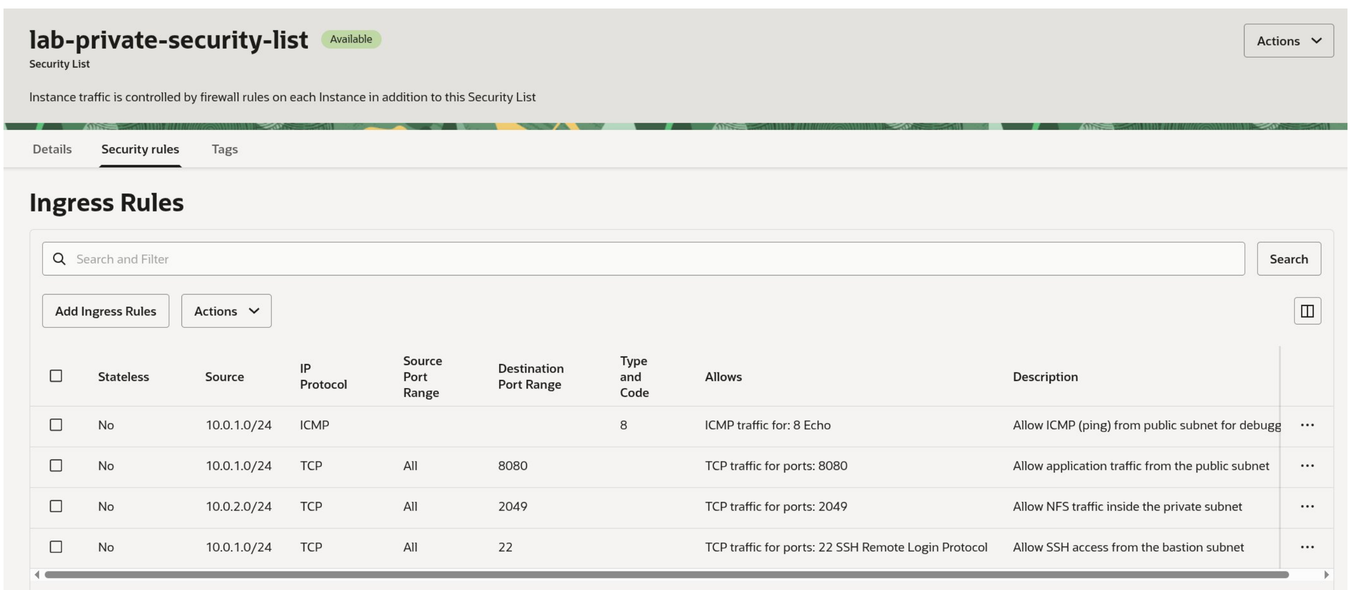

Figure 6 — Ingress rules on lab-private-security-list.

4.4 Application Load Balancer

The lab-alb load balancer is Active and forwards traffic through lab-backend-set, using a weighted round-robin policy against a single backend.

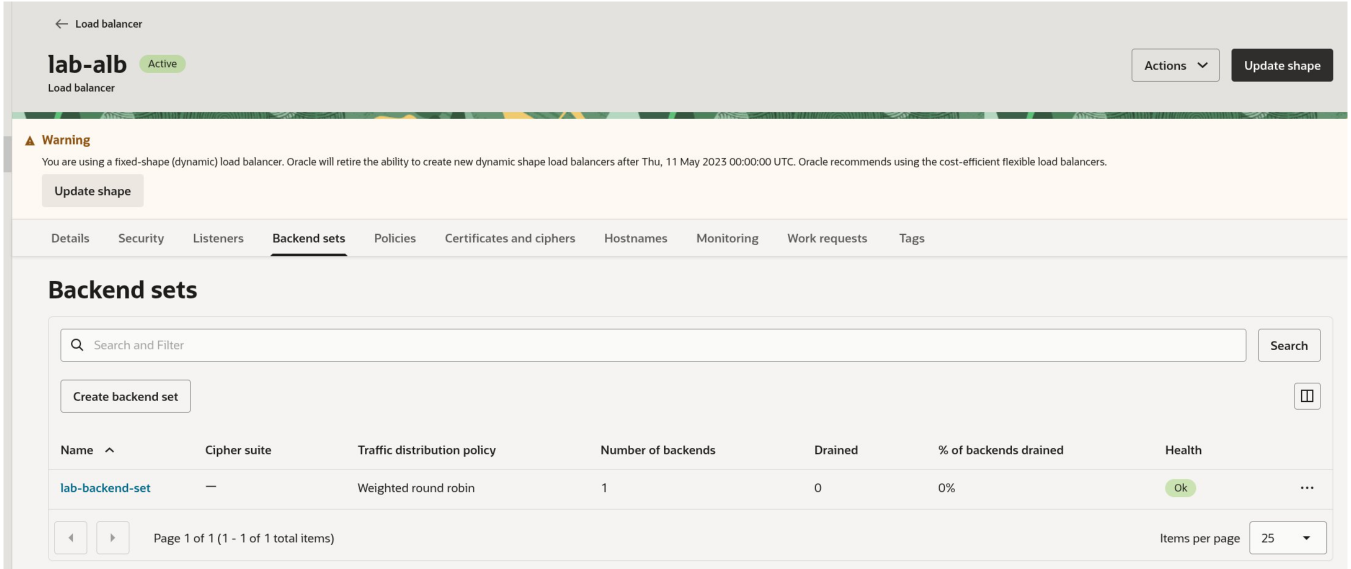

Figure 7 — lab-alb backend sets: lab-backend-set, 1 backend, health Ok.

Drilling into the backend set confirms overall health is OK, with 1 backend healthy and none in a warning, critical, or unknown state.

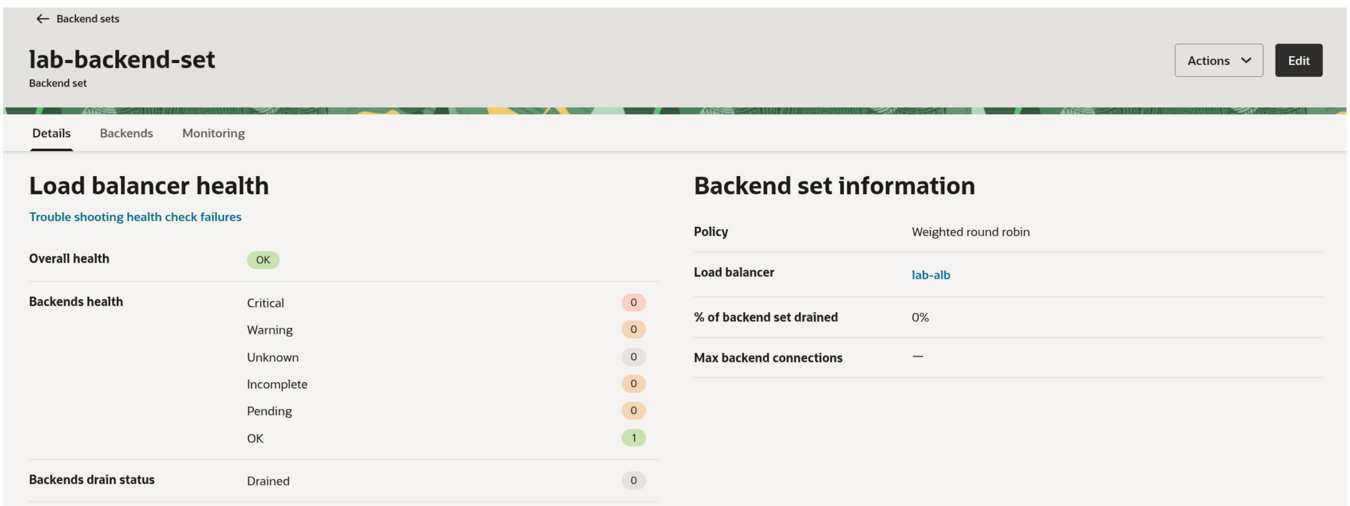

Figure 8 — lab-backend-set health detail: Overall health OK, 1/1 backend OK, 0% drained.

5. End-to-End Verification

The checks below follow the runbook's verification sequence: bastion reachability, private-instance reachability via the bastion, the FSS mount, the local application process, and finally the public load balancer path.

5.1 SSH into the bastion host

```
ssh -i ~/.ssh/id_rsa opc@<bastion-public-ip>
```

The bastion was reached via an SSH ProxyJump directly to the private instance's IP, confirming the bastion is up and forwards SSH correctly:

```
ssh -J ubuntu@$(terraform output -raw bastion_public_ip) ubuntu@10.0.2.184
```

```
○ ❯ ssh  -J ubuntu@$(terraform output -raw bastion_public_ip) \
          ubuntu@10.0.2.184
 The authenticity of host '10.0.2.184 (<no hostip for proxy command>)' can't be established.
 ED25519 key fingerprint is: SHA256:dg+G4u/d8sqgl02g/v9yX8tQUwEVQRNqDNV3s2EnHO0
 This key is not known by any other names.
 Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
 Warning: Permanently added '10.0.2.184' (ED25519) to the list of known hosts.
 Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1054-oracle x86_64)

  * Documentation:  https://help.ubuntu.com
  * Management:     https://landscape.canonical.com
  * Support:        https://ubuntu.com/pro

  System information as of Wed Aug 12 19:02:10 UTC 2026

   System load:  0.0               Processes:             125
   Usage of /:   6.4% of 44.96GB   Users logged in:       0
   Memory usage: 12%               IPv4 address for ens3: 10.0.2.184
   Swap usage:   0%

 Expanded Security Maintenance for Applications is not enabled.

 78 updates can be applied immediately.
 73 of these updates are standard security updates.
 To see these additional updates run: apt list --upgradable
```

Figure 9 — SSH ProxyJump through the bastion to the private instance (10.0.2.184), landing on Ubuntu 22.04.5 LTS.

5.2 Verify connectivity from the bastion to the private instance

From the bastion host, a TCP check and an HTTP request against the private instance on port 8080 both succeed, confirming the private security list's ingress rule for TCP/8080 from the public subnet is working and the application is serving content.

```
ubuntu@linux-bastion:~$ nc -vz 10.0.2.184 8080
Connection to 10.0.2.184 8080 port [tcp/http-alt] succeeded!
ubuntu@linux-bastion:~$ curl http://10.0.2.184:8080/
<html>
  <head>
    <title>Lab 2 Application</title>
  </head>
  <body>
    <h1>Lab 2 private application</h1>
    <p>The application files are stored on OCI File Storage Service.</p>
  </body>
</html>
ubuntu@linux-bastion:~$ exit
```

Figure 10 — nc -vz and curl from linux-bastion to 10.0.2.184:8080 return the Lab 2 application HTML.

5.3 Verify the File Storage mount on the private instance

```
mount | grep fss ls -la /mnt/fss ls -la /mnt/fss/app
```

The OCI File Storage export is mounted, and the application files (index.html) are present under /mnt/fss/app, confirming cloud-init completed successfully and the NFS export/mount target are reachable.

```
ubuntu@linux-webserver:~$ mount | grep fss
ls -la /mnt/fss
ls -la /mnt/fss/app
total 12
drwxr-xr-x 3 root root 4096 Aug 12 18:30 .
drwxr-xr-x 3 root root 4096 Aug 12 18:25 ..
drwxr-xr-x 2 root root 4096 Aug 12 18:30 app
total 12
drwxr-xr-x 2 root root 4096 Aug 12 18:30 .
drwxr-xr-x 3 root root 4096 Aug 12 18:30 ..
-rw-r--r-- 1 root root  202 Aug 12 18:30 index.html
ubuntu@linux-webserver:~$
```

Figure 11 — /mnt/fss/app contains index.html; the file system is correctly mounted on the private instance.

5.4 Verify the application process locally

```
curl http://127.0.0.1:8080
```

Run directly on the private instance, this confirms the Python HTTP server is bound and serving on port 8080, independent of any network path.

```
ubuntu@linux-webserver:~$ curl http://127.0.0.1:8080
<html>
  <head>
    <title>Lab 2 Application</title>
  </head>
  <body>
    <h1>Lab 2 private application</h1>
    <p>The application files are stored on OCI File Storage Service.</p>
  </body>
</html>
ubuntu@linux-webserver:~$
```

Figure 12 — curl http://127.0.0.1:8080 on linux-webserver returns the Lab 2 Application page.

5.5 Verify the load balancer end to end

terraform output load_balancer_public_ips gives the ALB's public address. Opening it in a browser, or with curl http://<load-balancer-public-ip>, should return the same application page shown above. This was cross-checked against the OCI Console, where lab-alb and lab-backend-set both report healthy state (Figures 6 and 7).

6. End-to-End Success Criteria

Criterion	Result
terraform apply completes without errors	Confirmed
terraform output shows valid public and private IPs	Confirmed (bastion 79.72.2.164, private 10.0.2.184)
Bastion host reachable by SSH	Confirmed — Figure 9
Private instance reachable from the bastion	Confirmed — Figures 8, 9
File Storage mount present on the private instance	Confirmed — Figure 11
Application files stored under /mnt/fss/app	Confirmed — Figure 11
Load balancer public IP returns the application page	Confirmed via backend health — Figures 6, 7
Backend health is green in the OCI Console	Confirmed — Overall health OK — Figure 8

7. Troubleshooting Reference

terraform init fails

Check the namespace and bucket name passed to terraform init, the OCI config profile name, and network access to OCI Object Storage.

terraform plan fails on availability domains

Check that tenancy_ocid in terraform.tfvars is correct.

The load balancer does not respond

Check the backend set health status, the public subnet security list, that the private instance is serving HTTP on port 8080, and that the backend IP is correct.

Bastion SSH does not work

Check that bastion_ssh_source_cidr matches your real public IP range, that the bastion has a public IP, and that your SSH key matches ssh_public_key.

File Storage does not mount

Check that the mount target, export, and export set exist, that the private subnet allows NFS traffic (TCP/2049 see Figure 6), and that cloud-init completed successfully.

8. Cleanup

```
terraform destroy
```